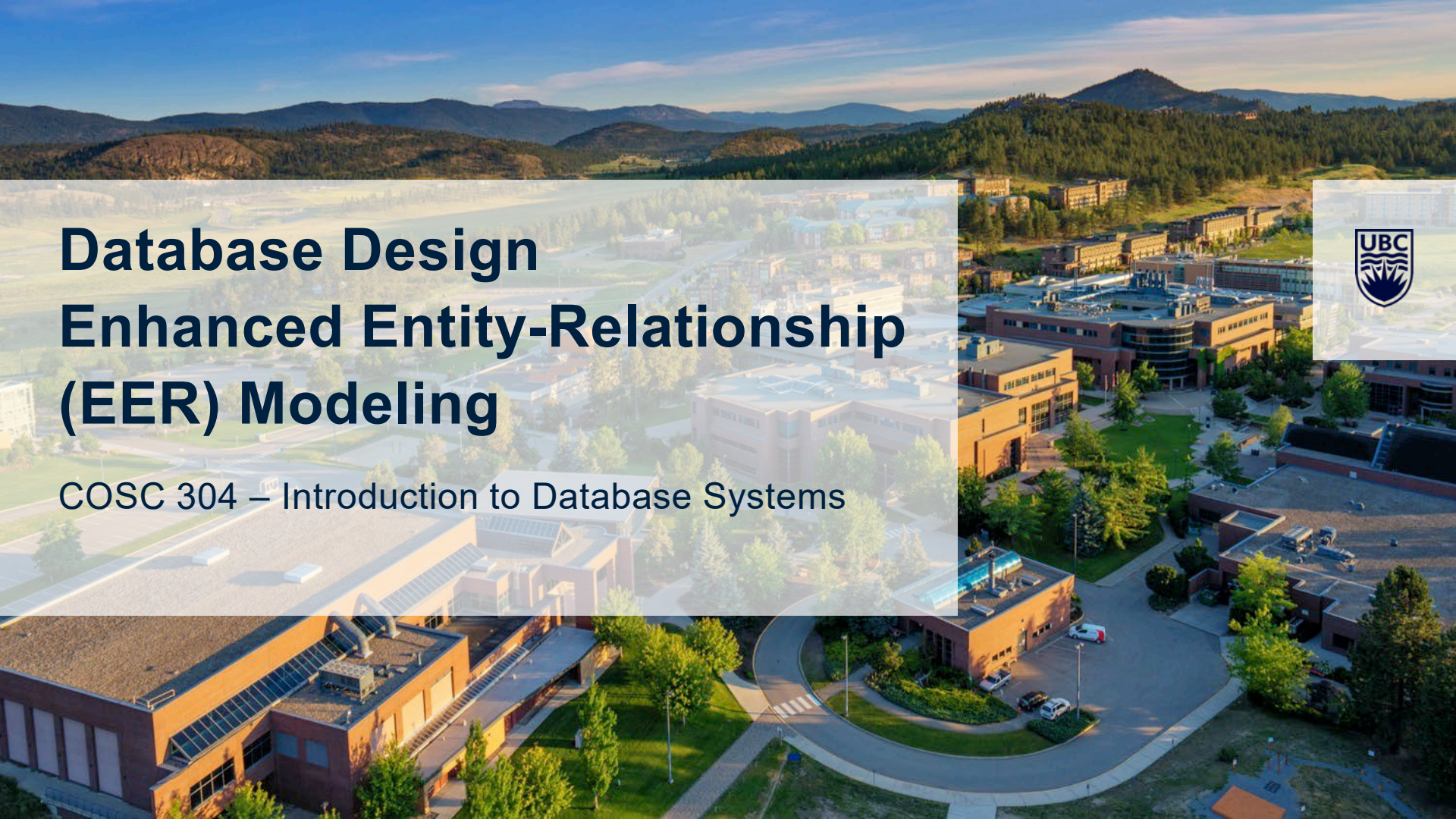




Database Design Enhanced Entity-Relationship (EER) Modeling

COSC 304 – Introduction to Database Systems



Enhanced Entity-Relationship Modeling

Enhanced Entity-Relationship (EER) modeling is an extension of ER modeling to include object-oriented concepts such as:

- superclasses and subclasses
- specialization and generalization
- aggregation and composition

These modeling constructs may allow more precise modeling of complex domains with natural subtype hierarchies such as:

- GIS (Geographical Information Systems)
- CAD/CAM systems (Computer-Aided Design/Manufacturing)

Review: Superclasses and Subclasses

The object-oriented ideas of inheritance and superclasses and subclasses are taught during programming in an OO language such as Java.

A **superclass** is a general class that is extended by one or more subclasses.

A **subclass** is a more specific class that extends a superclass by inheriting its methods and attributes and then adding its own methods and attributes.

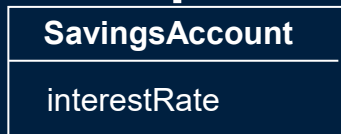
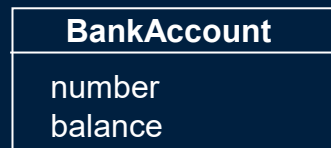
Inheritance is the process of a subclass inheriting all the methods and attributes of a superclass.

Superclasses and Subclasses Example

Java code:

```
public class SavingsAccount extends BankAccount
```

UML class diagram:

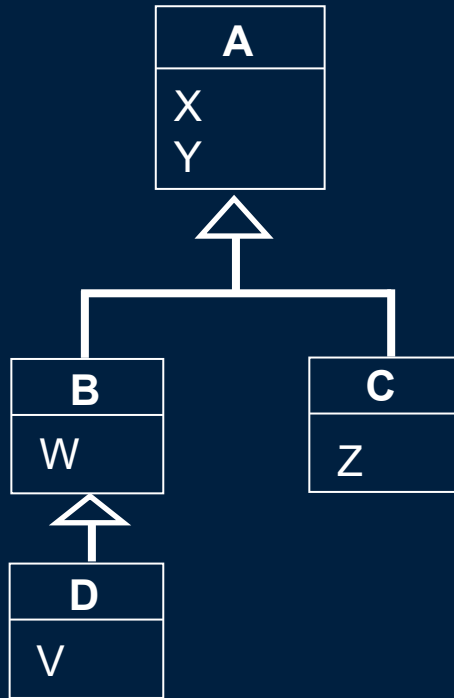


Triangle points to superclass

SavingsAccount **IS-A** BankAccount
SavingsAccount **inherits** number **and** balance

Superclasses and Subclasses Question

Question: How many of the following statements are **true**?



- 1) D is a superclass.
- 2) D has 1 attribute (including inherited attributes).
- 3) B and C are subclasses of A.
- 4) B inherits V from D.
- 5) D inherits attribute X.

A) 0 **B) 1** **C) 2** **D) 3** **E) 4**

When to use EER Modeling?

Many database projects do not need the object-oriented modeling features of EER modeling.

EER modeling is useful when the domain being modeled is object-oriented in nature and inheritance simplifies the design. Some cases:

- 1) *Attribute inheritance* can avoid storing many subclass-specific attributes as NULL in a single relation.
- 2) Subclasses can explicitly model subsets of an entity type that have their own attributes or participate in their own relationships.

When to use EER Modeling?

Using Attribute Inheritance



Emp Relation

<u>eno</u>	ename	bdate	title	salary	supereno	dno
E1	J. Doe	01-05-75	EE	30000	E2	null
E2	M. Smith	06-04-66	SA	50000	E5	D3
E3	A. Lee	07-05-66	ME	40000	E7	D2
E4	J. Miller	09-01-50	PR	20000	E6	D3
E5	B. Casey	12-25-71	SA	50000	E8	D3
E6	L. Chu	11-30-65	EE	30000	E7	D2
E7	R. Davis	09-08-77	ME	40000	E8	D1
E8	J. Jones	10-11-72	SA	50000	null	D1

Note that the `title` attribute indicates what job the employee does at the company. Consider if each job title had its own unique information that we would want to record such as:

- EE, PR - programming language used (`lang`), DB used (`db`)
- SA, ME - MBA? (`MBA`), `bonus`

When to use EER Modeling?

Using Attribute Inheritance (2)



We could represent all these attributes in a single relation:

eno	ename	bdate	title	salary	supereno	dno	lang	db	MBA	bonus
E1	J. Doe	01-05-75	EE	30000	E2		C++	MySQL		
E2	M. Smith	06-04-66	SA	50000	E5	D3			N	2000
E3	A. Lee	07-05-66	ME	40000	E7	D2			N	3000
E4	J. Miller	09-01-50	PR	20000	E6	D3	Java	Oracle		
E5	B. Casey	12-25-71	SA	50000	E8	D3			Y	4000
E6	L. Chu	11-30-65	EE	30000	E7	D2	C++	DB2		
E7	R. Davis	09-08-77	ME	40000	E8	D1			N	3000
E8	J. Jones	10-11-72	SA	50000		D1			Y	6000

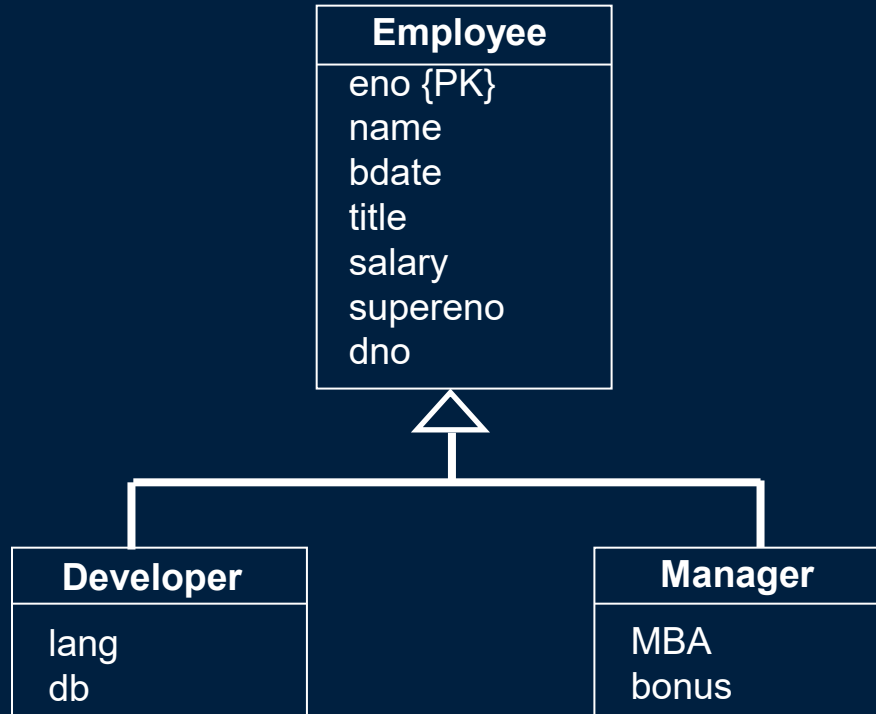
Many attributes do not apply to every employee, resulting in numerous NULL values and a less precise model.

When to use EER Modeling?

Using Attribute Inheritance (3)



An EER design can represent these as two subclasses of Employee called Developer and Manager:



When to use EER Modeling?

Using Attribute Inheritance (4)



Resulting relations:

Employee Relation

<u>eno</u>	ename	bdate	title	salary	supereno	dno
E1	J. Doe	01-05-75	EE	30000	E2	null
E2	M. Smith	06-04-66	SA	50000	E5	D3
E3	A. Lee	07-05-66	ME	40000	E7	D2
E4	J. Miller	09-01-50	PR	20000	E6	D3
E5	B. Casey	12-25-71	SA	50000	E8	D3
E6	L. Chu	11-30-65	EE	30000	E7	D2
E7	R. Davis	09-08-77	ME	40000	E8	D1
E8	J. Jones	10-11-72	SA	50000	null	D1

Developer Relation

<u>eno</u>	lang	db
E1	C++	MySQL
E4	Java	Oracle
E6	C++	DB2

Manager Relation

<u>eno</u>	MBA	bonus
E2	N	2000
E3	N	3000
E5	Y	4000
E7	N	3000
E8	Y	6000

Generalization and Specialization

Subclasses and superclasses are created by using either generalization or specialization.

Specialization is a top-down process that starts with a superclass and defines more specific subclasses.

- The specialized classes may contain their own attributes. Attributes common to all subclasses remain in the superclass.

Generalization is a bottom-up process that starts with several related entity types and identifies a common superclass.

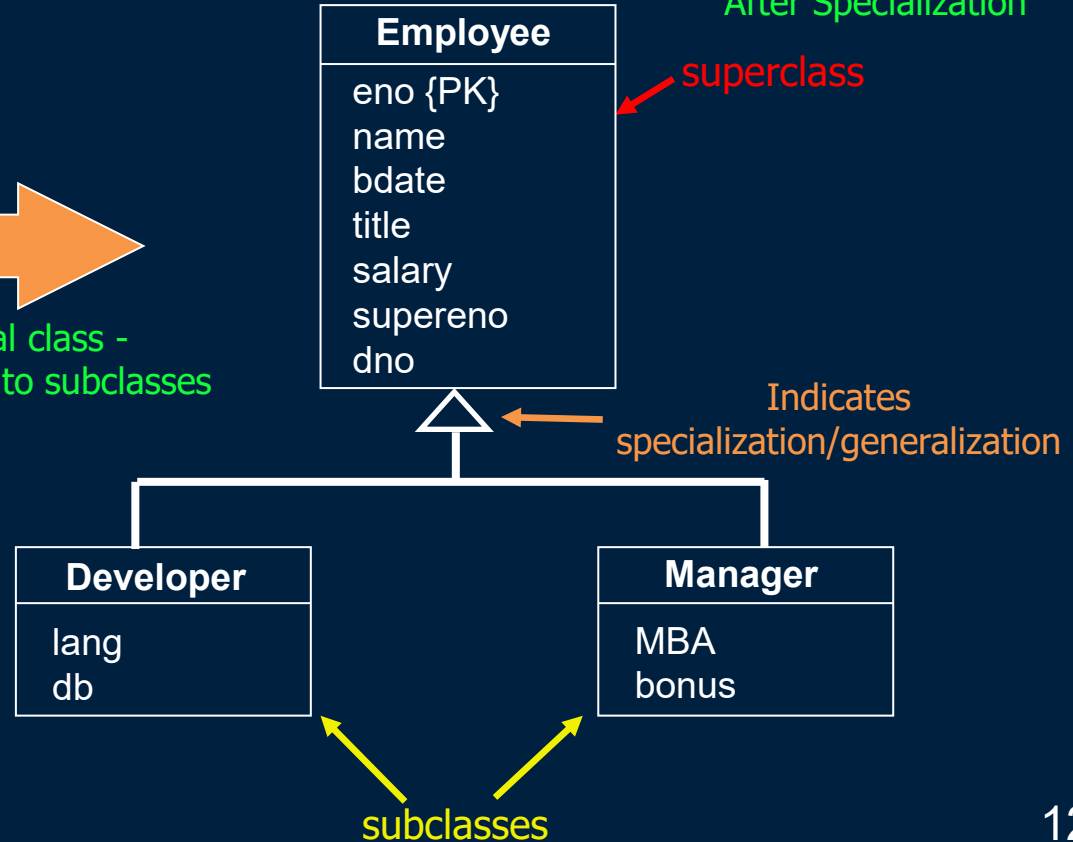
Specialization Example

Before Specialization

Employee
eno {PK}
name
bdate
title
salary
supereno
dno
lang
db
MBA
bonus

General class -
specialize into subclasses

After Specialization



Generalization Example

Developer
eno {PK}
name
bdate
title
salary
supereno
dno
lang
db

Manager
eno {PK}
name
bdate
title
salary
supereno
dno
MBA
bonus

Before
Generalization



Specific classes -
generalize to create
superclass

Employee
eno {PK}
name
bdate
title
salary
supereno
dno

After
Generalization

superclass



Indicates
specialization/generalization

Developer
lang
db

Manager
MBA
bonus

subclasses



Constraints on Generalization and Specialization

Participation constraint - determines if every member in a superclass must participate as a member of one of its subclasses.

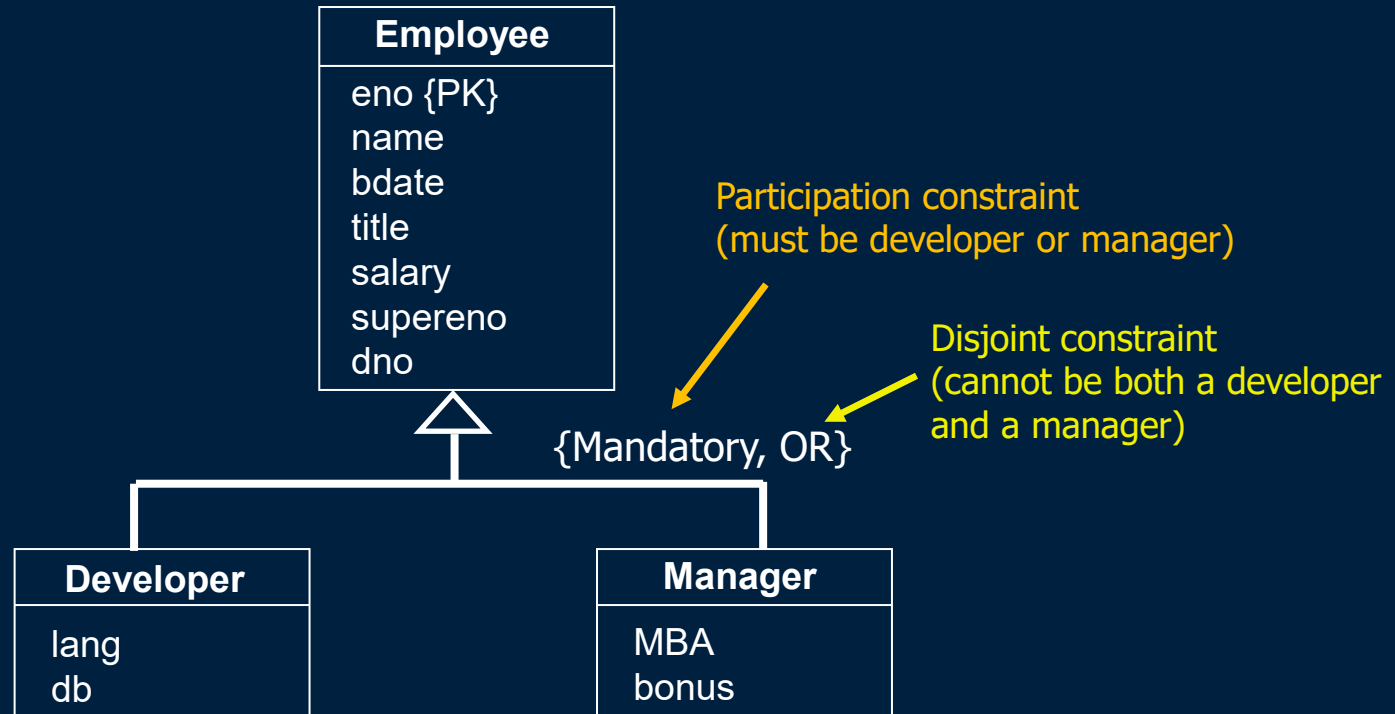
- **Mandatory**: every superclass instance belongs to at least one subclass
- **Optional**: some superclass instances may belong to no subclass

Disjoint constraint - determines if a member of a superclass can be a member of one or more of its subclasses.

- **OR (disjoint)**: an instance may belong to at most one subclass
- **AND (overlapping)**: an instance may belong to multiple subclasses

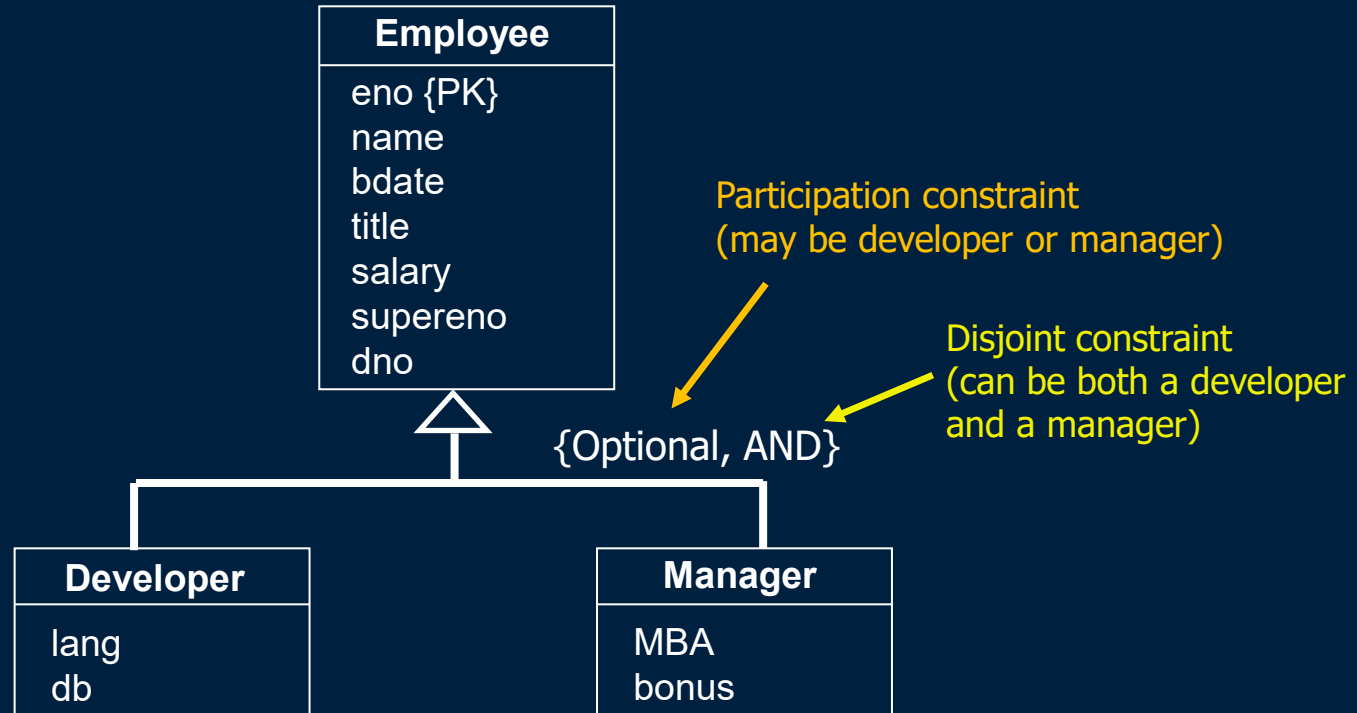
Constraints Example: MANDATORY, OR

An employee must be either a developer or a manager, but cannot be both.



Constraints Example: OPTIONAL, AND

An employee may be a Developer, a Manager, both, or neither.



Predicate-Defined Subclasses

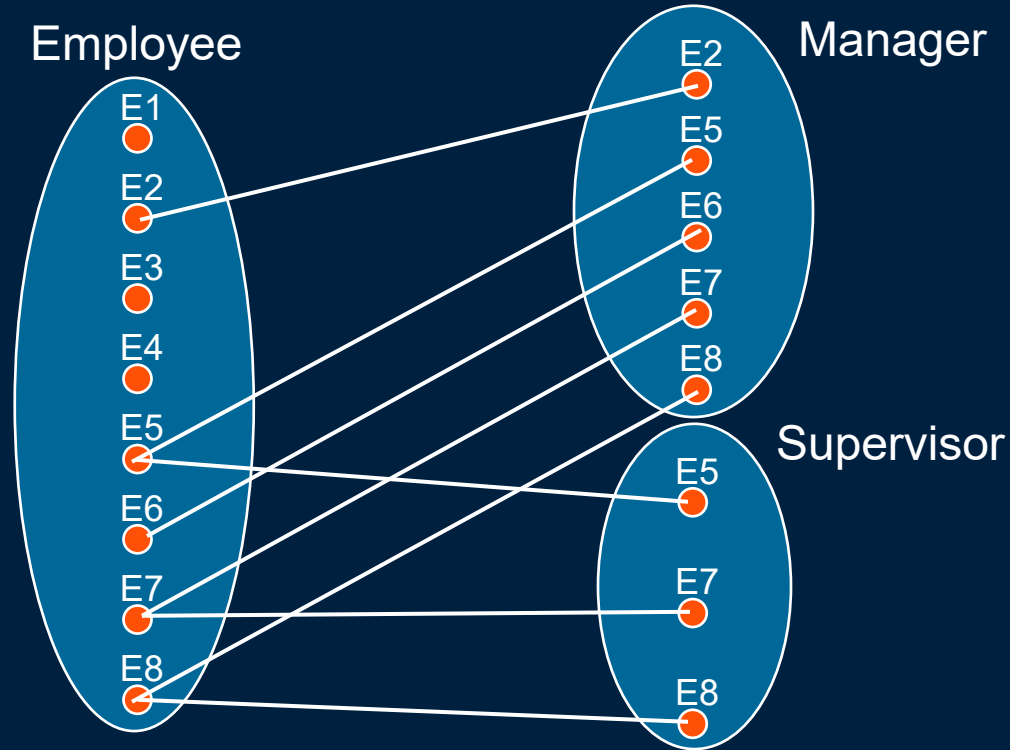
Predicate-defined subclasses use a rule to determine subclass membership.

- For example, a subclass called `RichEmployees` can be defined with a membership predicate such as `salary > 100000`.

Attribute-defined subclasses are a special case where subclass membership is determined by the value of one or more attributes.

- For example, the `title` field could be used as a *defining attribute* for the `Developer` and `Manager` subclasses.
 - Emp is in `Developer` if `title = 'EE' or 'PR'`
 - Emp is in `Manager` if `title = 'ME' or 'SA'`

Constraints Question



Note: Which participation and disjoint constraints for superclass Employee (with subclasses Manager and Supervisor) are consistent with these instances?

Relationship Constraints vs. Inheritance Constraints



There is a parallel between relationship constraints on relationships and inheritance constraints on superclasses and subclasses.

- Minimum # of occurrences – called participation constraint in both cases
- Maximum # of occurrences – Cardinality limits the maximum number of related entities, while the disjoint constraint determines whether an instance may belong to one or multiple subclasses.

Possible combinations:

<u>Subclass Constraints</u>	<u>Relationship Constraints</u>
Optional, AND	0..*
Optional, OR	0..1
Mandatory, AND	1..*
Mandatory, OR	1..1

EER Question

Question: How many of the following statements are **true**?

- 1) Generalization is a bottom-up process.
- 2) In an UML diagram, the inheritance arrow points towards the superclass.
- 3) OPTIONAL and MANDATORY are possible choices for the participation constraint.
- 4) If the disjoint constraint is AND, a given object can be a member of multiple subclasses.
- 5) If the participation constraint is OPTIONAL, the disjoint constraint must be AND.

A) 0 B) 1 C) 2 D) 3 E) 4

Multiple Inheritance

If each class only has one superclass, then the class diagram is said to be a *specialization* or *type hierarchy*.

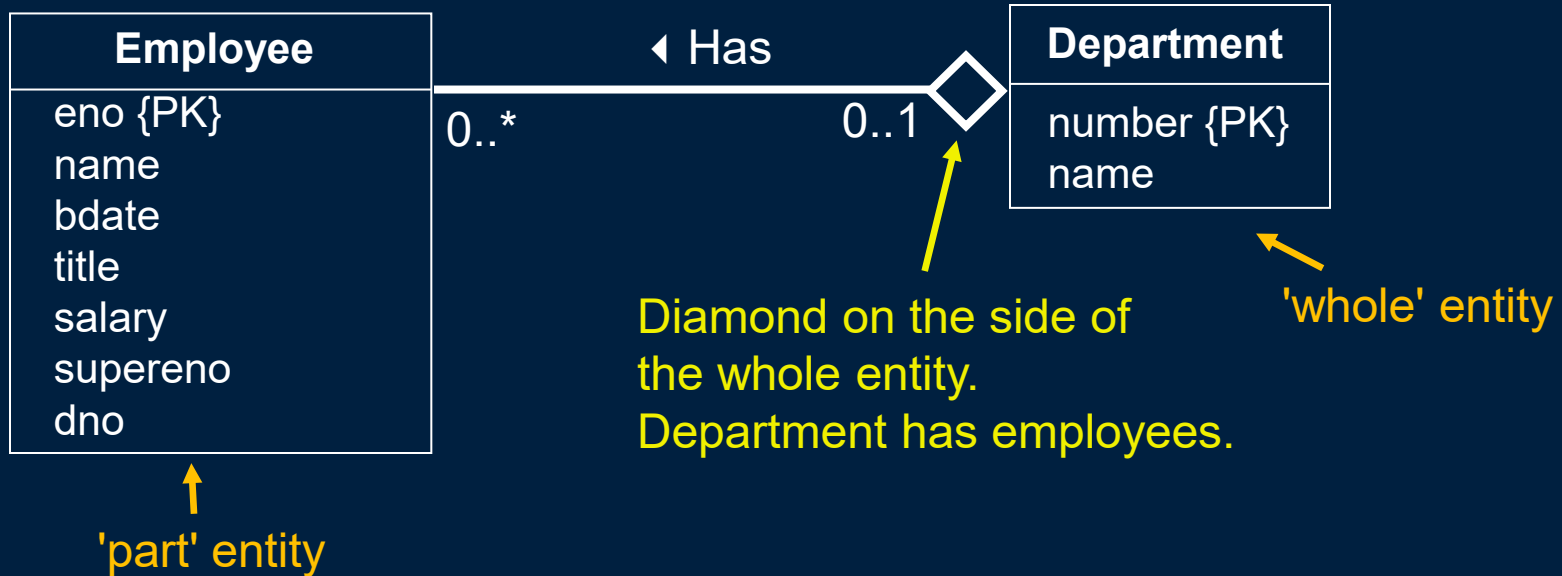
If a class may have more than one superclass, then the class diagram is said to be a *specialization* or *type lattice*.

Multiple inheritance can model domains where a subclass naturally belongs to more than one superclass, but it increases model complexity.

Aggregation

Aggregation is UML notation for a whole-part (HAS-A / PART-OF) association. One entity type is the *whole*, the other is the *part*.

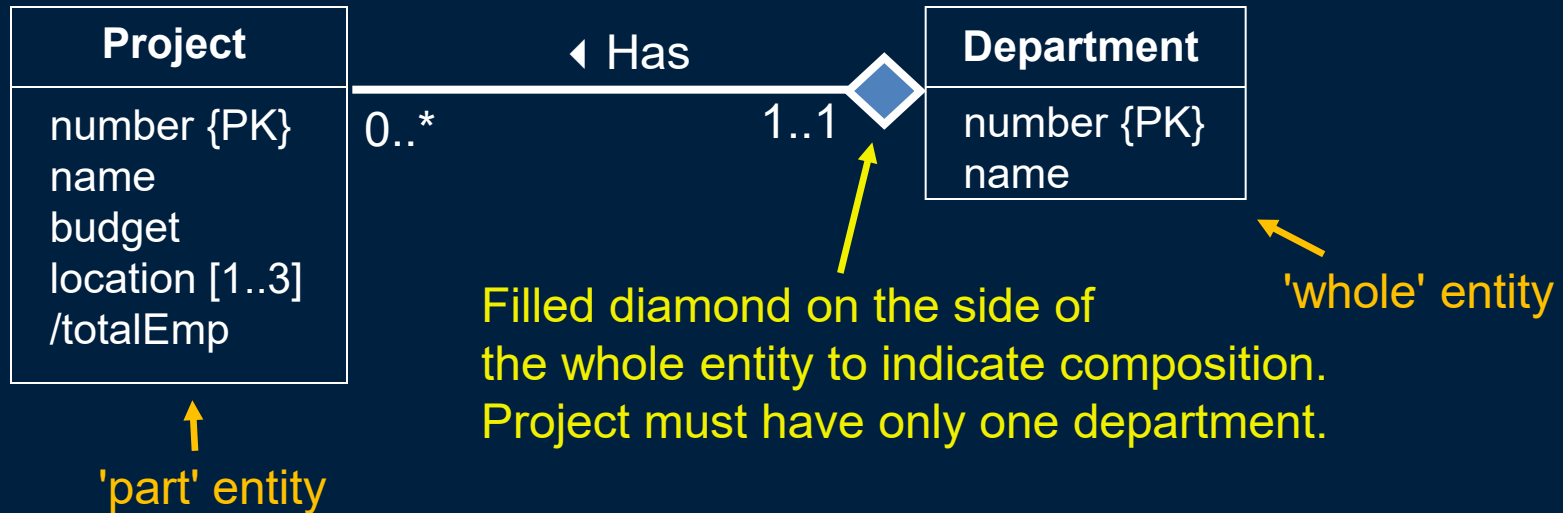
Example:



Composition

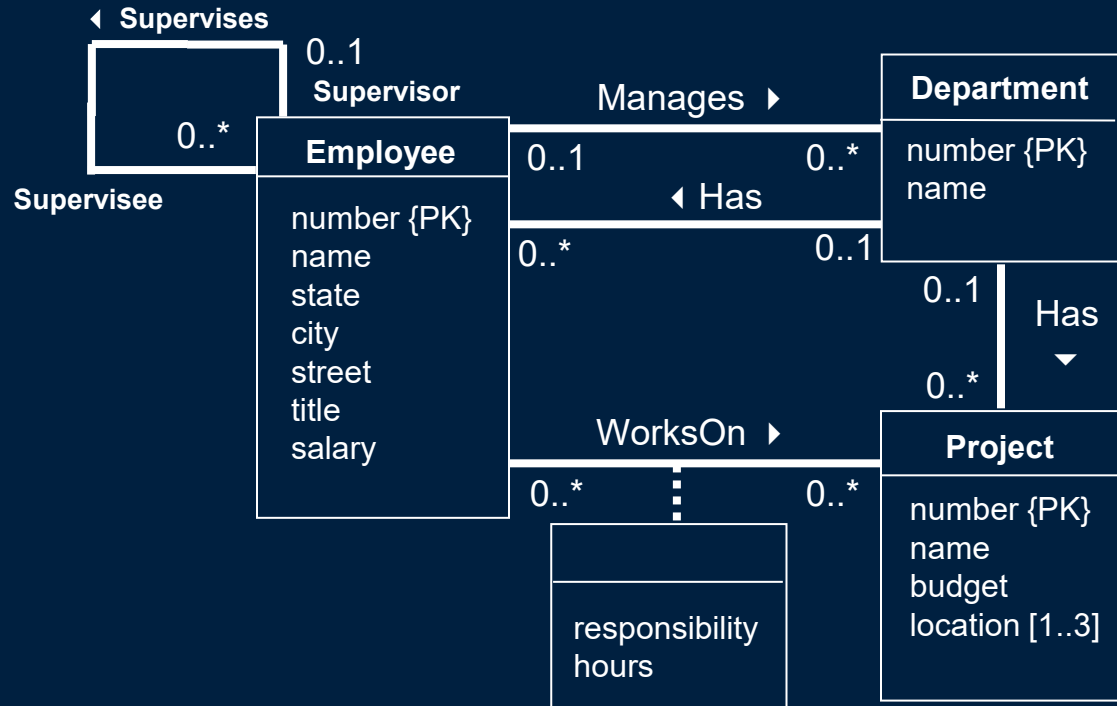
Composition is a strong whole–part association in which the part belongs to exactly one whole and its lifetime depends on that whole.

Example:

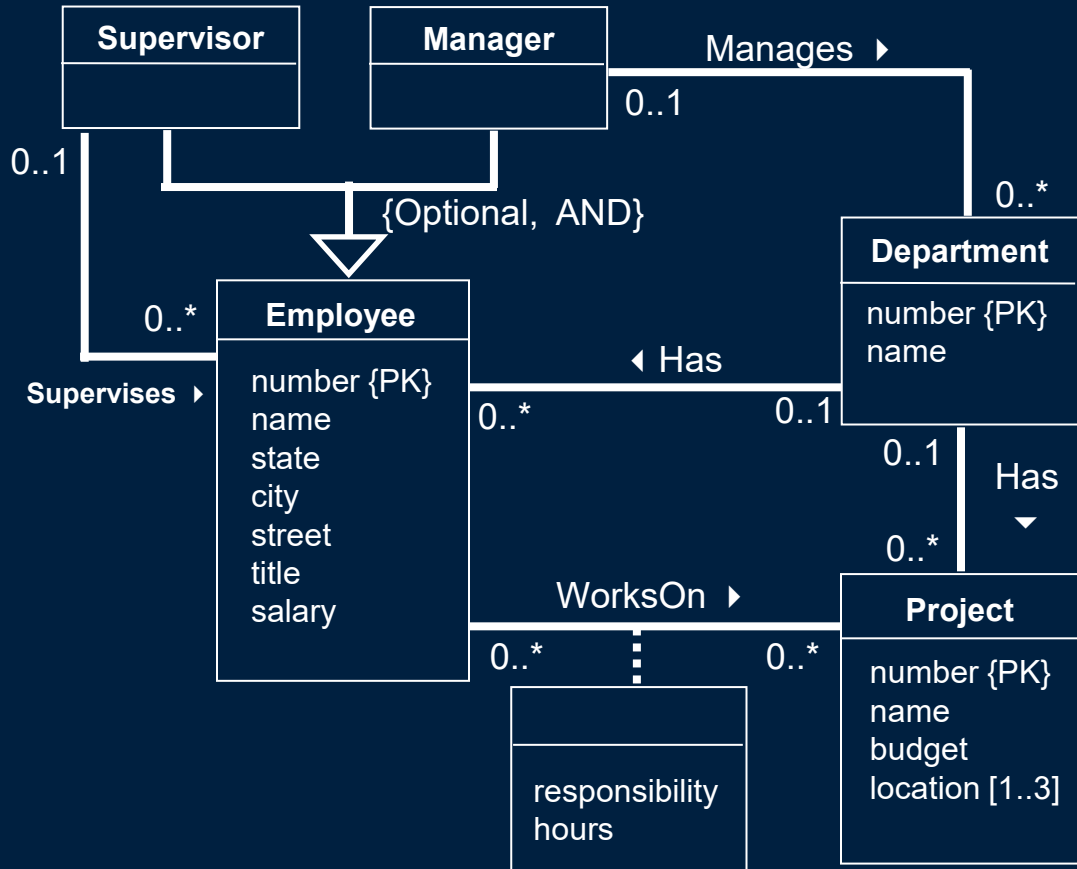


Note: The min-max constraint on the whole side of the relationship (in this case department) must always be 1..1 when modeling composition. Why?

Original ER Model Example



EER Model Example



Conclusion

The **Enhanced Entity-Relationship (EER)** model allows for object-oriented design features to be captured.

Generalization and **specialization** are two complementary processes for constructing superclasses and subclasses.

- A subclass **IS-A** superclass and inherits its attributes and relationships.

Participation and **disjoint constraints** apply to subclasses.

- Participation of a superclass may be mandatory or optional in a subclass.
- A superclass may only be a member of one subclass (disjoint constraint indicated by OR) or multiple (indicated by AND).

Aggregation and composition are used to model HAS-A or PART-OF relationships.

Many database designs can be modeled adequately without EER constructs. EER is useful when subtype or whole-part semantics improve the design.

Objectives

Explain the IS-A relationship between a subclass and superclass and identify inherited attributes and relationships.

Given an EER diagram, identify superclasses/subclasses and determine the attributes and relationships inherited by each subclass.

Explain the difference between the participation constraint and the disjoint constraint.

Recognize aggregation and composition notation and explain their basic difference.



THE UNIVERSITY OF BRITISH COLUMBIA

